



University of Stuttgart
Collaborative
Artificial Intelligence



e l l i s
European Laboratory for Learning and Intelligent Systems



University of Stuttgart
Germany

Machine Perception and Learning for Collaborative Intelligent Systems

Prof. Dr. Andreas Bulling

WS 2025/2026

Collaborative Artificial Intelligence Group, University of Stuttgart

www.collaborative-ai.org 

Interactive Machine Learning

- Semi-supervised Learning
- Active Learning
- Other Interesting AL Techniques Used in Practice
- Graph-based Active and Semi-supervised Methods
- Deep Active Learning

Generative Models I

Introduction

Mixture of Gaussians

Autoencoders

Variational Autoencoders

Generative Models I

Introduction

- O. Hilliges @ ETHZ
- Paul Liang and Louis-Philippe Morency @ CMU

- Supervised Learning
- Data: $(\mathbf{x}, \mathbf{y})$ where $\mathbf{x}$ are data points and $\mathbf{y}$ is the label
- Goal: Learn a mapping from $\mathbf{X} \rightarrow \mathbf{y}$
- Example tasks: image classification, regression

- **Unsupervised Learning**
- Data: X where x are data points
- No y , i.e. no labels
- Goal: Learn an underlying structure of data
- Example tasks: clustering

- Given training data, we want to learn a “model” of this data and be able to sample from the same distribution



Training samples $\sim p_{data}(x)$



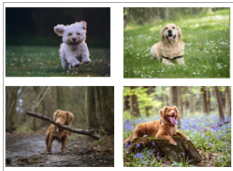
Generated samples $\sim p_{model}(x)$

Source: Otmar Hilliges, ETHZ

- Goal: We want $p_{model}(x)$ to be similar to $p_{data}(x)$

- Learn $p_{model}(x)$ where x can be text, images, videos, or multimodal data
- Given x , evaluate $p_{model}(x)$ - realistic data should have high $p_{model}(x)$ and vice versa
- Sample new x according to $p_{model}(x)$ - e.g. want to sample realistic images

- Unsupervised representation learning - learn what the data have in common (e.g. features such as ears, tail, etc.)



INPUT (x)	RECONSTRUCTION (AUR)	RECONSTRUCTION (Gen-RNN)
unable to stop herself, she briefly, gently, touched his hand.	unable to stop herself, she leaned forward, and touched his eyes.	unable to help her , and her back and her into my way.
why didn't you tell me?	why didn't you tell me?	why didn't you tell me?"
a strange glow of sunlight shines down from above, paper white and blinding, with no heat.	the light of the sun was shining through the window, illuminating the room.	a tiny light on the door, and a few inches from behind him out of the door.
he handed her the slip of paper.	he handed her a piece of paper.	he took a sip of his drink.

Source: Paul Liang, CMU

- Sometimes we also want $p(x|c)$ - **conditional generation**, where c represents a category (e.g. faces, outdoor scenes) from which we want to sample images
- We might want $p(x_2|x_1, c)$ - **style transfer**, where c is a stylistic change and x_1 is another image



Source: Paul Liang, CMU

- Sometimes we also want $p(x|c)$ - **conditional generation**, where c represents a category (e.g. faces, outdoor scenes) from which we want to sample images
- We might want $p(x_2|x_1, c)$ - **style transfer**, where c is a stylistic change and x_1 is another image

From negative to positive

consistently slow .
consistently good .
consistently fast .

my goodness it was so gross .
my husband 's steak was phenomenal .
my goodness was so awesome .

it was super dry and had a weird taste to the entire slice .
it was a great meal and the tacos were very kind of good .
it was super flavorful and had a nice texture of the whole side .

Source: Paul Liang, CMU

- A bit of brainstorming...

- Progress in generating face images



Source: Paul Liang, CMU

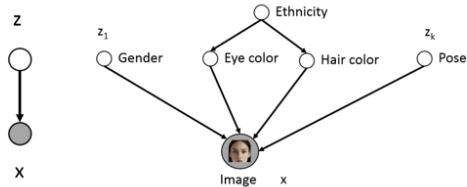
[Goodfellow et al., 2014; Radford et al., 2016; Karras et al., 2018, 2019]

- Lots of variability in images x due to gender, eye colour, hair colour, pose, etc.
- Unless images are annotated, these factors of variation are not explicitly available = latent
- Idea: explicitly model these factors using latent variables z



Source: Paul Liang, CMU

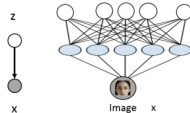
- Only shaded variables x are observed in the data



Source: Paul Liang, CMU

- Latent variables z are unobserved - correspond to high-level features
- Idea: explicitly model these factors using latent variables z
 - We want z to represent useful features e.g. hair color, pose
 - But very difficult to specify these conditionals by hand and they're unobserved
 - Solution: Try to learn them

- Put a prior on z : $z \sim \mathcal{N}(0, I)$



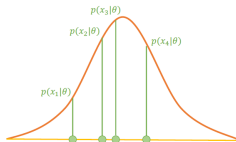
Source: Paul Liang, CMU

$p(x|z) = \mathcal{N}(\mu_\theta(z), \Sigma_\theta(z))$ where μ_θ and Σ_θ are neural networks

- Hope that after training, z will correspond to meaningful latent factors of variation - useful features for unsupervised representation learning
- Given a new image x , features can be extracted via $p(z|x)$

Likelihood and Maximum Likelihood (MLE)

- Likelihood is a function of the model parameters
- MLE extremely successful and widely used: e.g. least squares and cross-entropy are MLE estimators

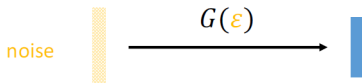


Source: Otmar Hilliges

$$L(\theta) = \prod_i p(x_i|\theta)$$

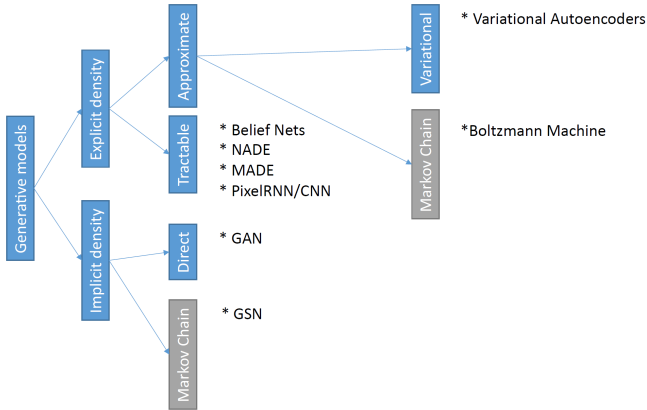
$$\log L(\theta) = \log \prod_i p(x_i|\theta)$$

$$\log L(\theta) = \sum_i \log p(x_i|\theta)$$



Source: Otmar Hilliges

- Highly expressive model class (universal)
- Density function not defined or intractable
- Lack of theory and learning algorithms compared to explicit models
- Basis for generative adversarial networks (GANs) and other recent models

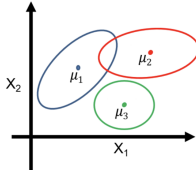


Source: Otmar Hilliges and Ian Goodfellow

Generative Models I

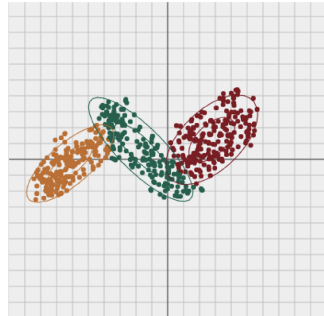
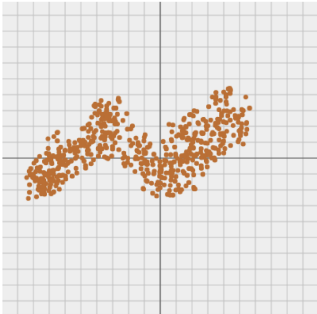
Mixture of Gaussians

- Mixture of Gaussians (Bayes network $z \rightarrow x$)
 $z \sim \text{Categorical}(1..K)$
 $p(x|z = k) = (\mu_k, \Sigma_k)$



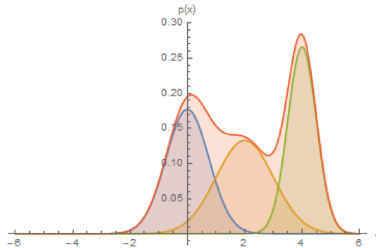
Source: Paul Liang, CMU

- Generative process:
 - Pick a mixture component by sampling z
 - Generate a data point by sampling from that Gaussian



Source: Paul Liang, CMU

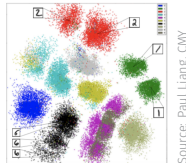
- Combining simple models into more expressive ones



Source: Paul Liang, CMU

$$p(x) = \sum_z p(x|z) = \sum_z p(z)p(x|z) = \sum_{k=1}^K p(z=k)\mathcal{N}(x; \mu_k, \Sigma_k)$$

- E.g. unsupervised clustering of digits
- Discovers clusters corresponding to factors of variation in the data
- Can generate new samples

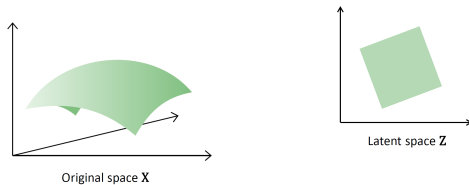


- **Cannot** learn features of data i.e. $p(z|x)$

Generative Models I

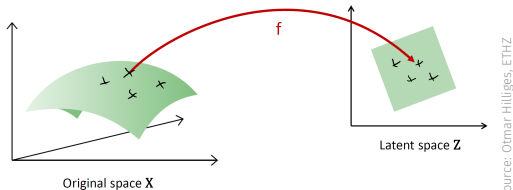
Autoencoders

- Autoencoder = encoder + decoder



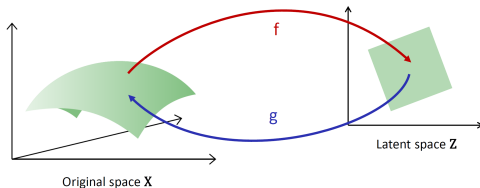
Source: Otmar Hilliges, ETHZ

- Autoencoder = encoder + decoder



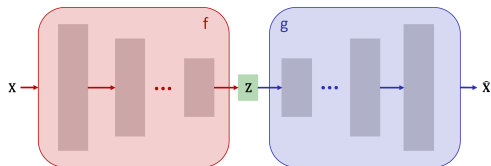
- The encoder f projects the original input space X into a latent space Z

- Autoencoder = encoder + decoder



Source: Otmar Hilliges, ETHZ

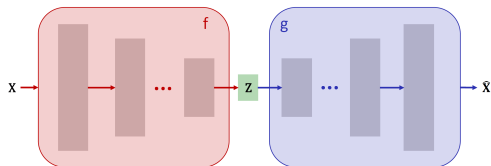
- The encoder f projects the original input space X into a latent space Z
- The decoder g maps samples from Z back to input space X
- $[g \circ f]$ approximates the identity function on the data



Source: Otmar Hilliges, ETHZ

- Given two parametrised mapping $f()$ and $g()$ training consists of minimising

$$\begin{aligned}\hat{\theta}_f, \hat{\theta}_g &= \arg \min_{\theta_f, \theta_g} \sum_n^N \|x_n - \hat{x}_n\|^2 \\ \hat{\theta}_f, \hat{\theta}_g &= \arg \min_{\theta_f, \theta_g} \sum_n^N \|x_n - g(f(x_n))\|^2\end{aligned}$$



Source: Otmar Hilliges, ETHZ

- A simple example of such an autoencoder would be with both f and g linear, in which case PCA gives the optimal solution.

X (original samples) $d = 1024$

7 2 1 0 4 1 4 9 5 9 0 6
9 0 1 5 9 7 3 4 9 6 6 5
4 0 7 4 0 1 3 1 3 4 7 2

$g \circ f(X)$ (CNN, $d = 2$)

7 2 1 0 4 1 4 9 5 9 0 6
9 0 1 5 9 7 3 4 9 6 6 5
4 0 7 4 0 1 3 1 3 3 7 2

$g \circ f(X)$ (PCA, $d = 2$)

9 2 1 0 9 1 9 9 0 9 0 0
9 0 1 3 9 9 0 9 9 0 9 0
9 0 9 9 0 1 3 1 3 0 9 0

X (original samples) $d = 1024$

7 2 1 0 4 1 4 9 5 9 0 6
9 0 1 5 9 7 3 4 9 6 6 5
4 0 7 4 0 1 3 1 3 4 7 2

$g \circ f(X)$ (CNN, $d = 4$)

7 2 1 0 4 1 4 9 5 9 0 6
9 0 1 5 9 7 3 4 9 6 6 5
4 0 7 4 0 1 3 1 3 0 7 2

$g \circ f(X)$ (PCA, $d = 4$)

9 2 1 0 9 1 9 9 0 9 0 0
9 0 1 3 9 9 0 9 9 0 9 0
9 0 9 9 0 1 3 1 3 4 9 0

Source: Otmar Hilliges, ETHZ

X (original samples) $d = 1024$

7 2 1 0 4 1 4 9 5 9 0 6
9 0 1 5 9 7 8 4 9 6 6 5
4 0 7 4 0 1 3 1 3 4 7 2

$g \circ f(X)$ (CNN, $d = 8$)

7 2 1 0 4 1 4 9 5 9 0 6
9 0 1 5 9 7 8 4 9 6 6 5
4 0 7 4 0 1 3 1 3 4 7 2

$g \circ f(X)$ (PCA, $d = 8$)

7 3 1 0 4 1 4 9 5 9 0 6
9 0 1 0 9 7 8 4 9 6 6 5
4 0 7 4 0 1 3 1 3 4 7 2

X (original samples) $d = 1024$

7 2 1 0 4 1 4 9 5 9 0 6
9 0 1 5 9 7 8 4 9 6 6 5
4 0 7 4 0 1 3 1 3 4 7 2

$g \circ f(X)$ (CNN, $d = 16$)

7 2 1 0 4 1 4 9 5 9 0 6
9 0 1 5 9 7 8 4 9 6 6 5
4 0 7 4 0 1 3 1 3 4 7 2

$g \circ f(X)$ (PCA, $d = 16$)

7 2 1 0 4 1 4 9 5 9 0 6
9 0 1 5 9 7 8 4 9 6 6 5
4 0 7 4 0 1 3 1 3 4 7 2

Source: Otmar Hilliges, ETHZ

- By introducing a simple density model (Gaussian) over the latent space Z the decoder g can be used to generate new data (e.g., images)

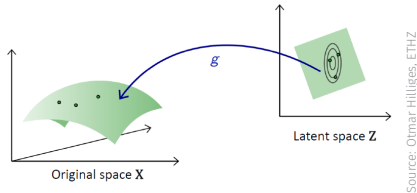
$$f(x) \sim \mathcal{N}(\hat{\mu}, \hat{\sigma}^2)$$

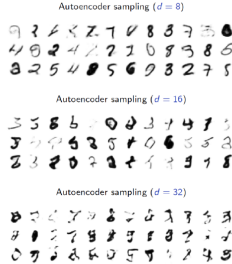
- where $\hat{\mu}$ and $\hat{\sigma}$ are mean and standard deviation on estimated training data.

- By introducing a simple density model (Gaussian) over the latent space Z the decoder g can be used to generate new data (e.g., images)

$$f(x) \sim \mathcal{N}(\hat{\mu}, \hat{\sigma}I)$$

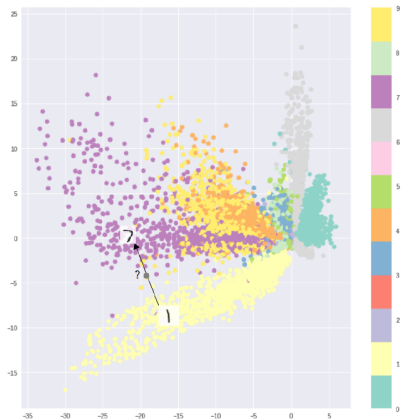
- where $\hat{\mu}$ and $\hat{\sigma}$ are mean and standard deviation on estimated training data.





Source: Otmar Hilliges, ETHZ

- The decoder is not able to generate good quality samples because the density model on the latent space Z is too simple and inadequate



Source: Otmar Hilliges, ETHZ

<https://towardsdatascience.com/>

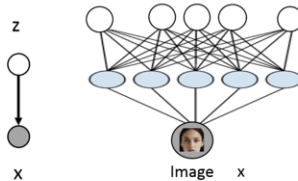
intuitively-understanding-variational-autoencoders-1bfe67eb5daf

Generative Models I

Variational Autoencoders

- Probabilistic version of autoencoders
- Allows for sampling from the learned model to generate new, unseen samples

- Put a prior on z : $z \sim \mathcal{N}(0, I)$



Source: Paul Liang, CMU

$p(x|z) = \mathcal{N}(\mu_{\theta}(z), \Sigma_{\theta}(z))$ where μ_{θ} and Σ_{θ} are neural networks

$p(x|z) = \mathcal{N}(\mu_\theta(z), \Sigma_\theta(z))$ where μ_θ and Σ_θ are neural networks

- Hope that after training, z will correspond to meaningful latent factors of variation - useful features for unsupervised representation learning
- Even though $p(x|z)$ is simple, marginal $p(x)$ is much richer/complex/flexible
- Given a new image x , features can be extracted via $p(z|x)$: natural for unsupervised learning tasks (clustering, representation learning, etc.)

- Learning parameters of VAE: we have a joint distribution $p(X, Z; \theta)$
- We have a dataset D where for each datapoint the x variables are observed (e.g. images, text) and the variables z are not observed (latent variables)
- We can try maximum likelihood estimation:

$$\log \prod_{x \in D} p(x; \theta) = \sum_{x \in D} \log p(x; \theta) = \sum_{x \in D} \log \sum_z p(x, z; \theta)$$

- Learning parameters of VAE: we have a joint distribution $p(X, Z; \theta)$
- We have a dataset D where for each datapoint the x variables are observed (e.g. images, text) and the variables z are not observed (latent variables)
- We can try maximum likelihood estimation:

$$\log \prod_{\mathbf{x} \in D} p(\mathbf{x}; \theta) = \sum_{\mathbf{x} \in D} \log p(\mathbf{x}; \theta) = \sum_{\mathbf{x} \in D} \underbrace{\log \sum_{\mathbf{z}} p(\mathbf{x}, \mathbf{z}; \theta)}$$

Need cheaper approximations to optimize for VAE parameters

intractable :-(
 - if z binary with 30 dimensions, need sum 2^{30} terms
 - if z continuous, integral is hard

Source: Paul Liang, CMU

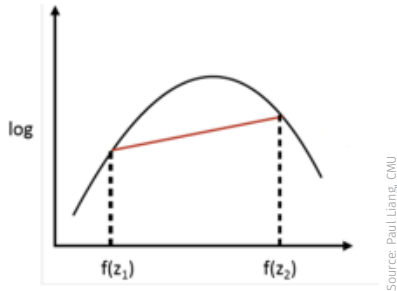
- Log-likelihood function with partially observed latent variables is hard to compute

$$\log\left(\sum_{z \in Z} p_{\theta}(x, z)\right) = \log\left(\sum_{z \in Z} \frac{q(z)}{q(z)} p_{\theta}(x, z)\right) = \log\left(\mathbb{E}_{z \sim q(z)} \left[\frac{p_{\theta}(x, z)}{q(z)}\right]\right)$$

$q(z)$ should be a simple distribution

- Concave functions

$$\log(px + (1 - p)x') \geq p \log(x) + (1 - p)\log(x')$$



- Use Jensen's inequality for concave functions:

$$\log(px + (1 - p)x') \geq p \log(x) + (1 - p)\log(x')$$

$$\log(\mathbb{E}_{z \sim q(z)}[f(z)]) = \log\left(\sum_z q(z)f(z)\right) \geq \sum_z q(z)\log(f(z))$$

Choosing $f(z) = \frac{p_\theta(x, z)}{q(z)}$

$$\log\left(\mathbb{E}_{z \sim q(z)}\left[\frac{p_\theta(x, z)}{q(z)}\right]\right) \geq \mathbb{E}_{z \sim q(z)}\left[\log\left(\frac{p_\theta(x, z)}{q(z)}\right)\right]$$

ELBO holds for any probability distribution $q(z)$ over latent variables:

$$\begin{aligned}\log p(x; \theta) &\geq \sum_z q(z) \log \left(\frac{p_\theta(x, z)}{q(z)} \right) \\ &= \sum_z q(z) \log p_\theta(x, z) - \sum_z q(z) \log(q(z))\end{aligned}$$

the last term represents the entropy $H(q)$ of q

$$= \sum_z q(z) \log p_\theta(x, z) + H(q)$$

Equality holds if $q(z) = p(z|x)$

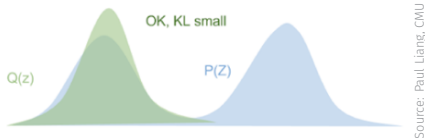
$$\log p(x; \theta) = \sum_z q(z) \log p_\theta(z, x; \theta) + H(q)$$

We want to choose $q(z)$ to be as close as possible to $p(z|x)$ and easy to compute

- The KL divergence for variational inference is:

$$D_{KL}(q(z)||p(z|x)) = \int q(z) \log \frac{q(z)}{p(z|x)} dz$$

- Intuitively, there are three cases
 - If q is low then we don't care (because of the expectation).
 - If q is high and p is high then we are happy.
 - If q is high and p is low then we pay a price.
- Note that p must be > 0 wherever $q > 0$



Starting from the KL divergence:

$$D_{KL}(q(z)||p(z|x)) = - \sum_z q(z) \log p(z, x; \theta) + \log p(x; \theta) - H(q) \geq 0$$

Re-derive ELBO from the KL divergence:

$$\log p(x; \theta) \geq \sum_z q(z) \log p(z, x; \theta) + H(q)$$

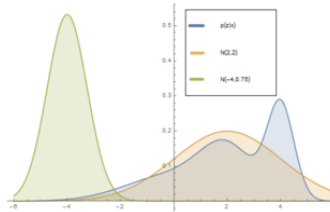
Equality holds if $q = p(z|x)$ because $D_{KL}(q||p) = 0$

$$\log p(x; \theta) = \sum_z q(z) \log p(z, x; \theta) + H(q)$$

In general:

$$\log p(x; \theta) = ELBO + D_{KL}(q(z) || p(z|x; \theta))$$

The closer the chosen q is to $p(z|x)$, the closer the ELBO is to the true likelihood

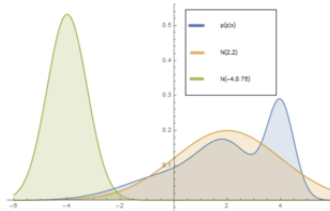


Source: Paul Liang, CMU

Suppose $q(z; \phi)$ is a (tractable) probability distribution over the hidden variables parametrised by ϕ (variational parameters)

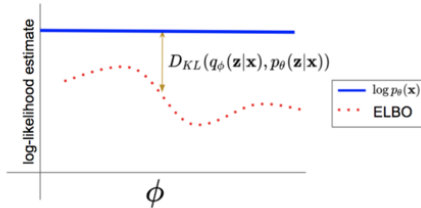
E.g. a Gaussian with a mean and covariance specified by ϕ

$$q(z; \phi) = \mathcal{N}(\phi_1, \phi_2)$$



Source: Paul Liang, CMU

- Variational inference: optimise variational parameters so that $q(z; \phi)$ is as close as possible to $p(z|x; \phi)$ while being simple to compute
- E.g. in figure, posterior (in blue) is better approximated by orange Gaussian than green



$$\begin{aligned}
 \log p(\mathbf{x}; \theta) &\geq \sum_{\mathbf{z}} q(\mathbf{z}; \phi) \log p(\mathbf{z}, \mathbf{x}; \theta) + H(q(\mathbf{z}; \phi)) = \underbrace{\mathcal{L}(\mathbf{x}; \theta, \phi)}_{\text{ELBO}} \\
 &= \mathcal{L}(\mathbf{x}; \theta, \phi) + D_{KL}(q(\mathbf{z}; \phi) \| p(\mathbf{z} | \mathbf{x}; \theta))
 \end{aligned}$$

Source: Paul Liang, CMU

- In practice how can we learn encoder parameters $p(\mathbf{z} | \mathbf{x}; \theta)$ and variational (decoder) parameters $q(\mathbf{z}; \phi)$ jointly

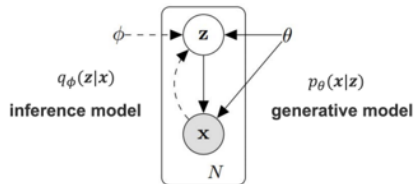


Figure courtesy: Kingma & Welling, 2014

Source: Paul Liang, CMU

$$\mathcal{L}(x; \theta, \phi) = E_{q_{\phi}(z|x)}[\log p(z, x; \theta) - \log q_{\phi}(z|x)]$$

$$= E_{q_{\phi}(z|x)}[\log p(z, x; \theta) - \log p(z) + \log p(z) - \log q_{\phi}(z|x)]$$

$$= E_{q_{\phi}(z|x)}[\log p(z, x; \theta)] - D_{KL}(q_{\phi}(z|x) || p(z))$$

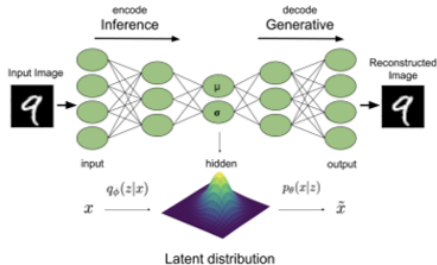
$$\begin{aligned}\mathcal{L}(\mathbf{x}; \theta, \phi) &= E_{q_{\phi}(\mathbf{z}|\mathbf{x})}[\log p(\mathbf{z}, \mathbf{x}; \theta) - \log q_{\phi}(\mathbf{z}|\mathbf{x})] \\ &= E_{q_{\phi}(\mathbf{z}|\mathbf{x})}[\log p(\mathbf{z}, \mathbf{x}; \theta) - \log p(\mathbf{z}) + \log p(\mathbf{z}) - \log q_{\phi}(\mathbf{z}|\mathbf{x})] \\ &= E_{q_{\phi}(\mathbf{z}|\mathbf{x})}[\log p(\mathbf{x}|\mathbf{z}; \theta)] - D_{KL}(q_{\phi}(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z}))\end{aligned}$$



Source: Paul Liang, CMU

What does the training objective $\mathcal{L}(\mathbf{x}; \theta, \phi)$ do?

- First term encourages $\hat{x} \approx x^i$ (x^i likely under $p(x|\hat{z}; \theta)$)
- Second term encourages $\hat{z}$ to be likely under the prior $p(z)$



Source: Paul Liang, CMU

- Take a data point x^i
- Map it to $\hat{z}$ by sampling from $q_\phi(z|x^i)$ (encoder)
- Reconstruct $\hat{x}$ by sampling from $p(x|\hat{z}; \theta)$ (decoder)

- Use backpropagation
- How do gradients flow through z ? Or any random operation requiring sampling?

$$\begin{aligned}\mathcal{L}(x; \theta, \phi) &= E_{q_\phi(z|x)}[\log p(z, x; \theta) - \log q_\phi(z|x)] \\ &= E_{q_\phi(z|x)}[\log p(z, x; \theta) - \log p(z) + \log p(z) - \log q_\phi(z|x)] \\ &= E_{q_\phi(z|x)}[\log p(x|z; \theta)] - D_{KL}(q_\phi(z|x) || p(z))\end{aligned}$$

We need to compute the gradients $\nabla_\theta \mathcal{L}(x; \theta, \phi)$ and $\nabla_\phi \mathcal{L}(x; \theta, \phi)$

$\nabla_\theta \mathcal{L}$ - easy how?

$$\begin{aligned}\mathcal{L}(x; \theta, \phi) &= E_{q_\phi(z|x)}[\log p(z, x; \theta) - \log q_\phi(z|x)] \\ &= E_{q_\phi(z|x)}[\log p(z, x; \theta) - \log p(z) + \log p(z) - \log q_\phi(z|x)] \\ &= E_{q_\phi(z|x)}[\log p(x|z; \theta)] - D_{KL}(q_\phi(z|x) || p(z))\end{aligned}$$

We need to compute the gradients $\nabla_\theta \mathcal{L}(x; \theta, \phi)$ and $\nabla_\phi \mathcal{L}(x; \theta, \phi)$

$$\begin{aligned}\nabla_\theta \mathcal{L}(x; \theta, \phi) &= \nabla_\theta E_{q_\phi(z|x)}[\log p(x|z; \theta)] - D_{KL}(q_\phi(z|x) || p(z)) \\ &= \nabla_\theta E_{q_\phi(z|x)}[\log p(x|z; \theta)]\end{aligned}$$

$$\begin{aligned}\mathcal{L}(x; \theta, \phi) &= E_{q_\phi(z|x)}[\log p(z, x; \theta) - \log q_\phi(z|x)] \\ &= E_{q_\phi(z|x)}[\log p(z, x; \theta) - \log p(z) + \log p(z) - \log q_\phi(z|x)] \\ &= E_{q_\phi(z|x)}[\log p(x|z; \theta)] - D_{KL}(q_\phi(z|x) || p(z))\end{aligned}$$

We need to compute the gradients $\nabla_\theta \mathcal{L}(x; \theta, \phi)$ and $\nabla_\phi \mathcal{L}(x; \theta, \phi)$

$$\begin{aligned}\nabla_\theta \mathcal{L}(x; \theta, \phi) &= \nabla_\theta E_{q_\phi(z|x)}[\log p(x|z; \theta)] - D_{KL}(q_\phi(z|x) || p(z)) \\ &= \nabla_\theta E_{q_\phi(z|x)}[\log p(x|z; \theta)] \\ &= E_{q_\phi(z|x)}[\nabla_\theta \log p(x|z; \theta)]\end{aligned}$$

$$\begin{aligned}\mathcal{L}(x; \theta, \phi) &= E_{q_\phi(z|x)}[\log p(z, x; \theta) - \log q_\phi(z|x)] \\ &= E_{q_\phi(z|x)}[\log p(z, x; \theta) - \log p(z) + \log p(z) - \log q_\phi(z|x)] \\ &= E_{q_\phi(z|x)}[\log p(x|z; \theta)] - D_{KL}(q_\phi(z|x) || p(z))\end{aligned}$$

We need to compute the gradients $\nabla_\theta \mathcal{L}(x; \theta, \phi)$ and $\nabla_\phi \mathcal{L}(x; \theta, \phi)$

$$\begin{aligned}\nabla_\theta \mathcal{L}(x; \theta, \phi) &= \nabla_\theta E_{q_\phi(z|x)}[\log p(x|z; \theta)] - D_{KL}(q_\phi(z|x) || p(z)) \\ &= \nabla_\theta E_{q_\phi(z|x)}[\log p(x|z; \theta)] \\ &= E_{q_\phi(z|x)}[\nabla_\theta \log p(x|z; \theta)] \\ &\approx \frac{1}{n} \sum_{i=1}^n \nabla_\theta \log p(x|z_i; \theta)\end{aligned}$$

$$\begin{aligned}\mathcal{L}(x; \theta, \phi) &= E_{q_\phi(z|x)}[\log p(z, x; \theta) - \log q_\phi(z|x)] \\ &= E_{q_\phi(z|x)}[\log p(z, x; \theta) - \log p(z) + \log p(z) - \log q_\phi(z|x)] \\ &= E_{q_\phi(z|x)}[\log p(x|z; \theta)] - D_{KL}(q_\phi(z|x) || p(z))\end{aligned}$$

We need to compute the gradients $\nabla_\theta \mathcal{L}(x; \theta, \phi)$ and $\nabla_\phi \mathcal{L}(x; \theta, \phi)$

$\nabla_\phi \mathcal{L}$ - **tricky** why?

The expectation also depends on ϕ

$$\begin{aligned}\mathcal{L}(x; \theta, \phi) &= E_{q_\phi(z|x)}[\log p(z, x; \theta) - \log q_\phi(z|x)] \\ &= E_{q_\phi(z|x)}[\log p(z, x; \theta) - \log p(z) + \log p(z) - \log q_\phi(z|x)] \\ &= E_{q_\phi(z|x)}[\log p(x|z; \theta)] - D_{KL}(q_\phi(z|x) || p(z))\end{aligned}$$

We need to compute the gradients $\nabla_\theta \mathcal{L}(x; \theta, \phi)$ and $\nabla_\phi \mathcal{L}(x; \theta, \phi)$

$\nabla_\phi \mathcal{L}$ - **tricky** why?

The expectation also depends on ϕ

$$\nabla_\phi \mathcal{L}(x; \theta, \phi) = \nabla_\phi E_{q_\phi(z|x)}[\log p(x|z; \theta)] - D_{KL}(q_\phi(z|x) || p(z))$$

We want to compute a gradient w.r.t ϕ for

$$E_{q(z;\phi)}[r(z)] = \int q(z; \phi) r(z) dz$$

where z is now continuous

We want to compute a gradient w.r.t ϕ for

$$E_{q(z;\phi)}[r(z)] = \int q(z;\phi)r(z)dz$$

where z is now continuous

Suppose $q(z;\phi) = \mathcal{N}(\mu, \sigma^2 I)$ is Gaussian with parameters $\phi = (\mu, \sigma)$. We have equivalent ways of sampling:

- Sample $z \sim q_\phi(z)$
- Sample $\epsilon \sim \mathcal{N}(0, I), z = \mu + \sigma\epsilon = g(\epsilon; \phi)$

Using the equivalence, we compute the expectation in two ways:

$$E_{z \sim q(x; \phi)}[r(z)] = E_{\epsilon \sim \mathcal{N}(0, I)}[r(g(\epsilon; \phi))] = \int p(\epsilon) r(\mu + \sigma \epsilon) d\epsilon$$

$$\nabla_{\phi} E_{q(z; \phi)}[r(z)] = \nabla_{\phi} E_{\epsilon}[r(g(\epsilon; \phi))] = E_{\epsilon}[\nabla_{\phi} r(g(\epsilon; \phi))]$$

Which can be estimated via Monte Carlo if r and g are differentiable w.r.t. ϕ and ϵ is easy to sample from (backpropagation)

$$E_{\epsilon}[\nabla_{\phi} r(g(\epsilon; \phi))] \approx \frac{1}{k} \sum_k \nabla_{\phi} r(g(\epsilon^k; \phi))$$

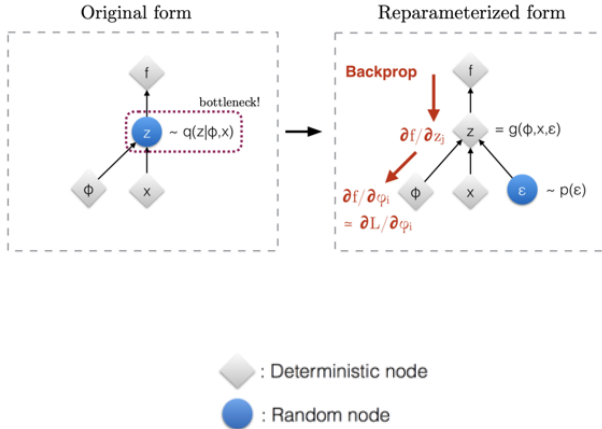
where $\epsilon^i \sim \mathcal{N}(0, I)$

$$\nabla_{\phi} \mathcal{L}(x; \theta, \phi) = \nabla_{\phi} E_{q_{\phi}(z|x)} [\log p(x|x; \theta) - D_{KL}(q_{\phi}(z|x) || p(z))]$$

$$\nabla_{\phi} E_{q_{\phi}(z|x)} [\log p(x|x; \theta)] = \nabla_{\phi} E_{\epsilon} [\log p(x|\mu + \sigma\epsilon; \theta)]$$

$$= E_{\epsilon} [\nabla_{\phi} \log p(x|\mu + \sigma\epsilon; \theta)]$$

$$\approx \frac{1}{n} \sum_{i=1}^n [\nabla_{\phi} \log p(x|\mu + \sigma\epsilon; \theta)]$$



Source: Paul Liang, CMU

$$\begin{aligned}
 \mathcal{L}(\mathbf{x}; \theta, \phi) &= E_{q_{\phi}(\mathbf{z}|\mathbf{x})}[\log p(\mathbf{z}, \mathbf{x}; \theta) - \log q_{\phi}(\mathbf{z}|\mathbf{x})] \\
 &= E_{q_{\phi}(\mathbf{z}|\mathbf{x})}[\log p(\mathbf{z}, \mathbf{x}; \theta) - \log p(\mathbf{z}) + \log p(\mathbf{z}) - \log q_{\phi}(\mathbf{z}|\mathbf{x})] \\
 &= E_{q_{\phi}(\mathbf{z}|\mathbf{x})}[\log p(\mathbf{x}|\mathbf{z}; \theta)] - D_{KL}(q_{\phi}(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z}))
 \end{aligned}$$

reconstruction
prior

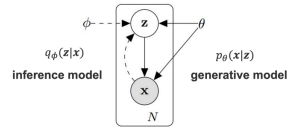
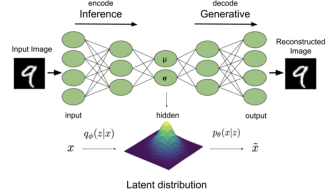


Figure courtesy: Kingma & Welling, 2014

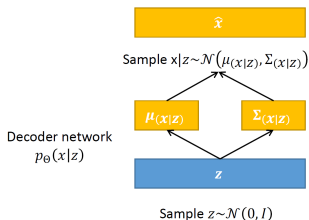
1. Take a datapoint $\mathbf{x}_i$.
2. Map it to μ, σ using $q_{\phi}(\mathbf{z}|\mathbf{x}_i)$. **encoder**
3. Sample $\epsilon \sim N(0, I)$ and compute $\hat{\mathbf{z}} = \mu + \sigma\epsilon$. **reparameterize**
4. Reconstruct $\hat{\mathbf{x}}$ by sampling from $p(\mathbf{x}|\hat{\mathbf{z}}; \theta)$. **decoder**

Differentiable using reparameterization trick

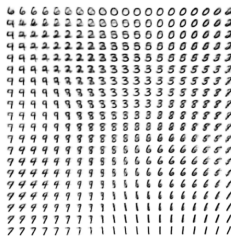


Source: Paul Liang, CMU

- Only need the decoder network at runtime
- Sample z from the prior

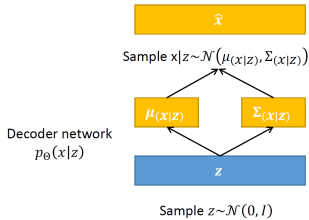


Source: Otmar Hilliges, ETHZ



Source: [Kingma and Welling, 2014]

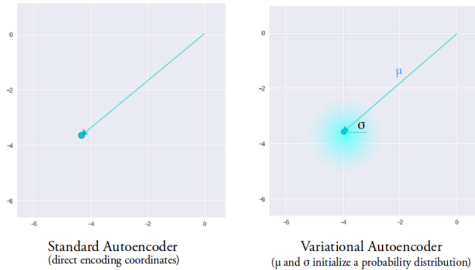
- Only need the decoder network at runtime
- Sample z from the prior



Source: Otmar Hilliges, ETHZ



Source: [Kingma and Welling, 2014]

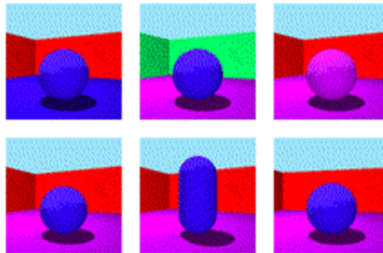


Source: Otmar Hilliges, ETHZ

<https://towardsdatascience.com/>

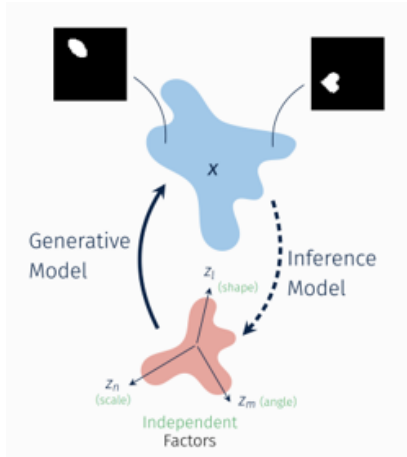
intuitively-understanding-variational-autoencoders-1bfe67eb5daf

- VAE for Disentangled representation learning
- Very useful for style transfer: disentangling style from content



disentanglement_lib

Source: Challenging Common Assumptions in the Unsupervised Learning of Disentangled Representations [Locatello et al., 2019]



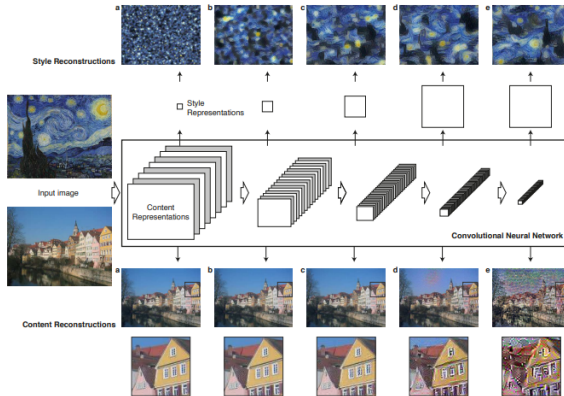
Source: Challenging Common Assumptions in the Unsupervised Learning of Disentangled Representations [Locatello et al., 2019]

Disentangled representation learning

- Very useful for style transfer: disentangling style from content

$$\mathcal{L}_\beta(x) = E_{q_\phi(z|x)}[\log p_\theta(x|z)] - \beta D_{KL}(q_\phi(z|x) || p(z))$$

- beta-VAE: $\beta = 1$ recovers VAE, $\beta > 1$ imposes a stronger constraint on the latent variables to have independent dimensions



Source: Image Style Transfer Using Convolutional Neural Networks [Gatys et al., 2016]

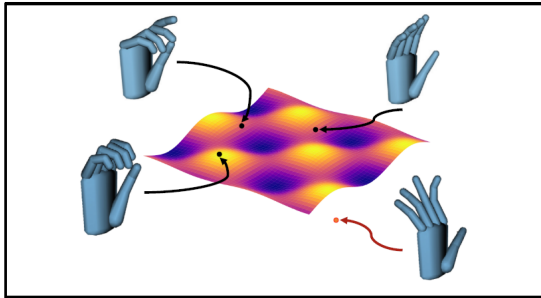


Source: Image Style Transfer Using Convolutional Neural Networks [Gatys et al., 2016]

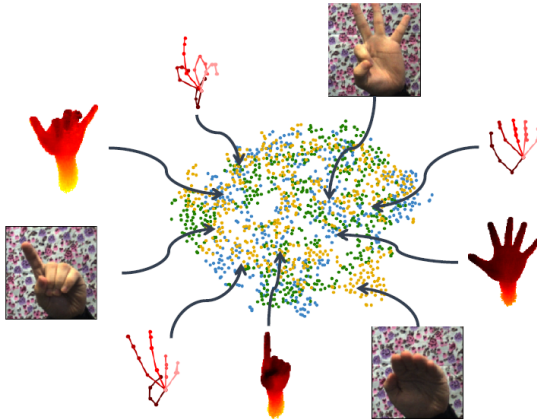
- VAE for Disentangled representation learning
- Very useful for style transfer: disentangling style from content

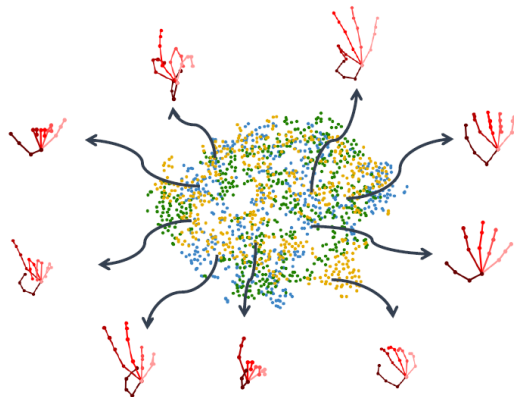
From negative to positive
consistently slow . consistently good . consistently fast .
my goodness it was so gross . my husband 's steak was phenomenal . my goodness was so awesome .
it was super dry and had a weird taste to the entire slice . it was a great meal and the tacos were very kind of good . it was super flavorful and had a nice texture of the whole side .

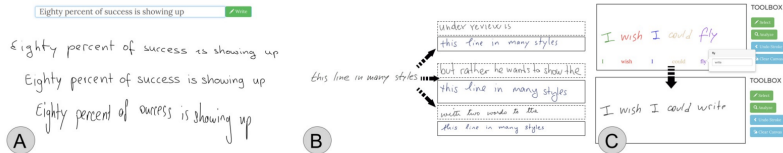
Source: Style Transfer from Non-Parallel Text by Cross-Alignment [Shen et al., 2017]



Source: Manifold of valid hand poses [Tagliasacchi et al., 2015]



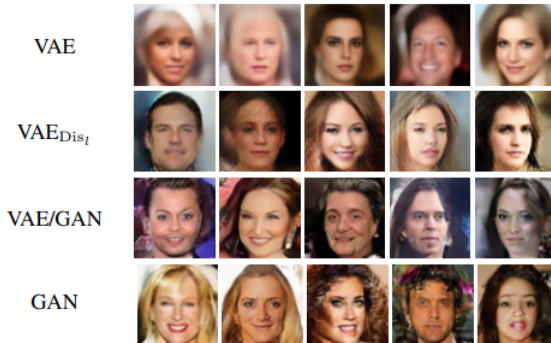




Source: [Aksan et al., 2018]

- Synthesize handwriting from typed text while giving users control over the visual appearance (A)
- Transfer style across handwriting samples (B)
- Edit handwritten samples at the word level (C)

- Relatively easier to train
- Explicit inference network $q(z|x)$
- More blurry images (due to reconstruction)



Source: Autoencoding beyond pixels using a learned similarity metric [Larsen et al., 2016]

Generative Models II

- Generative Adversarial Networks (GANs)
- Extensions
- Applications
- Case Study

- E. Aksan, F. Pece, and O. Hilliges. Deepwriting: Making digital ink editable via deep generative modeling. *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, 2018.
- L. A. Gatys, A. S. Ecker, and M. Bethge. Image style transfer using convolutional neural networks. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2414–2423, 2016.
- I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. C. Courville, and Y. Bengio. Generative adversarial nets. *Advances in Neural Information Processing Systems (NeurIPS)*, 2014.
- T. Karras, T. Aila, S. Laine, and J. Lehtinen. Progressive growing of gans for improved quality, stability, and variation. *arXiv preprint arXiv:1710.10196*, 2018.
- T. Karras, S. Laine, and T. Aila. A style-based generator architecture for generative adversarial networks. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4396–4405, 2019.
- D. P. Kingma and M. Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2014.

- A. B. L. Larsen, S. K. Sønderby, H. Larochelle, and O. Winther. Autoencoding beyond pixels using a learned similarity metric. *arXiv preprint arXiv:1512.09300v2*, 2016.
- F. Locatello, S. Bauer, M. Lucic, S. Gelly, B. Schölkopf, and O. Bachem. Challenging common assumptions in the unsupervised learning of disentangled representations. *arXiv preprint arXiv:1811.12359v4*, 2019.
- A. Radford, L. Metz, and S. Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. *arXiv preprint arXiv:1511.06434v2*, 2016.
- T. Shen, T. Lei, R. Barzilay, and T. Jaakkola. Style transfer from non-parallel text by cross-alignment. *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- A. Tagliasacchi, M. Schröder, A. Tkach, S. Bouaziz, M. Botsch, and M. Pauly. Robust articulated icp for real-time hand tracking. *Computer Graphics Forum*, 34, 2015.